

Unit - II

Process Management :->

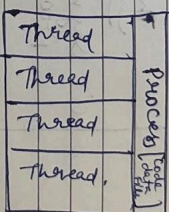
Program:- when it is not executing it is just a program sitting there (in memory) and doing nothing.

But the moment it begins execution, at that time we call it as a process.

Introduction of Process Management:-

- > Process management is a core function of an operating system by dealing with creating, scheduling and coordinating processes to ensure efficient CPU utilization & smooth system performance.
- > Single-tasking systems are easy to manage since only one process runs at a time.
- > Multiprogramming / multitasking systems are more complex, as multiple processes need to share the CPU efficiently.
- > Active processes may share memory and other resources, requiring careful management.

=> Thread:- A Thread is the unit of execution within a process. A process can have any number of threads. Just one thread to many threads.



When a program begins its execution, it becomes the process. Within this process there are certain units of execution, each unit is known as a thread.

-> In OS, threads are needed with a process to improve performance, responsiveness and efficient resource utilization.

Fig. - Ex:-

When you write a program, you write it in high level language, and then the compiler will compile your code into binary form. In order to run it, the OS will load it into memory, and when it is executing, we call it a process.

→ A program can have multiple process.
 → A thread is a unit of execution of a process. One process can have one or multiple threads.

Process

(i) Program in execution.

(ii) More time to terminate, creation & context switching.

(iii) less efficient communication.

(iv) Process does not share memory.

(v) Heavy weight

(vi) Context switching is slow

(vii) Independent

(viii) Blocking a process will not block another process.

Thread

(i) Segment of process.

(ii) less time.

(iii) more efficient.

(iv) Thread share memory.

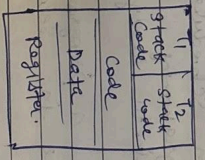
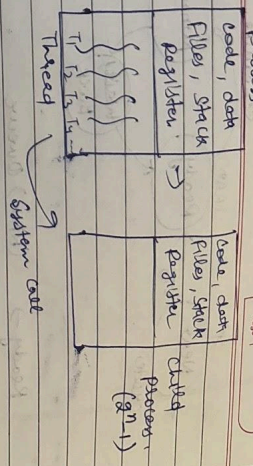
(v) Light weight

(vi) Fast

(vii) Interdependent

(viii) Blocking a thread will block entire process.

Process

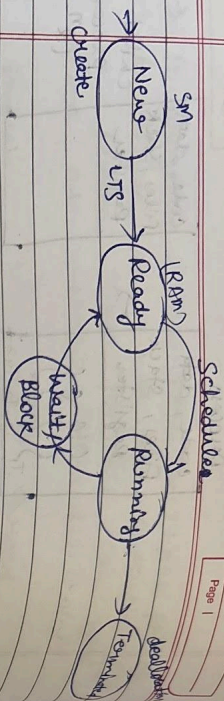


Process state

A process in an OS passes through multiple states as it begins execution, waits for resources, gets scheduled, runs and terminates.
 → These states collectively describe the complete life cycle of a process.

Process life cycle

Dispatch /



Ready → (RAM) Queue
LTS → long term Scheduler

(multiprogramming) more process come in to Ready state

→ Create:- You create a program (in) and store in Secondary memory.

→ Means it is staying in a stable state, that's called a new state.

OR you just open an Application. Then a lot of process starts running at the background, so in actual

it's a new state. Means there were no process before

→ that, but now we created a new process

→ Right After creating we just poked it, now it come in an active state

→ Ready:- Queue, means the process come in RAM.

→ Now it come in fast memory before that it was staying - stable.

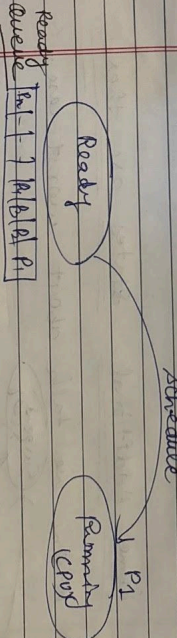
→ Each process wants its execution and get CPU for some time.

→ Running:- All the process are schedules on running state

→ How many process are there in running state depends upon the no. of CPU.

→ Because running state means we are executing that process.

→ In uni-processor system there is only one CPU.



→ Now CPU starts its execution

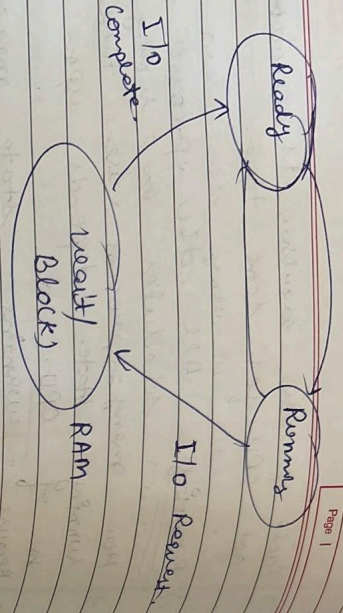
→ Now process completed / terminated.

→ Now the process deallocates the memory.

→ when process come in Ready state in RAM we provided it some address and resources.

(short term Scheduler) (Pick the process from Ready state and move it to Running state)

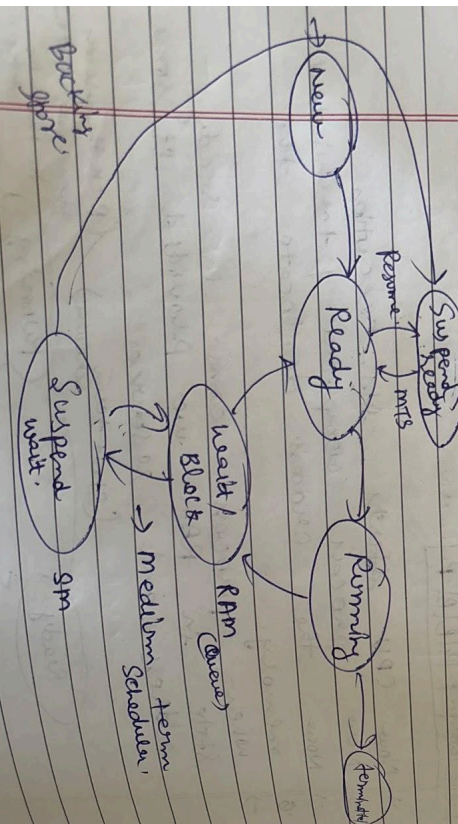
(Priority + Time Quantum) (multiplexing) (CPU execute it a fixed amount of time)



→ If the process wants to access some I/O devices, then it comes into wait or block state.

→ Some Additional states are there.

→ When we talk about worst case.



→ Whenever the processes want to access some I/O devices.

→ Then all processes are sent to the wait state after that wait state is full. because it is placed in RAM and there is limited memory space.

→ So all processes are jump into suspend wait that is placed in secondary memory.

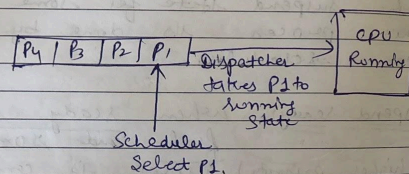
→ After the completion of I/O devices operations, the processes will come back to wait state if there is empty space.

→ Otherwise, all processes wait in suspend state for some time. This process is done by medium term scheduler.

Suspend ready: when ready state is full and some VIP (higher priority) process is come then some processes shift to the suspend ready state.

Process Scheduling:-

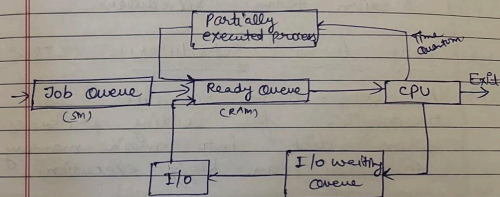
- Scheduling is important in OS with multiple processes might be eligible for running at a time.
- It determines the order in which process are executed by the CPU.
- It involving selecting process from a process scheduling queue and allocating CPU time to them according to specific scheduling algorithm.



⇒ Process Scheduling Queue:-

- (1) Job Queue:- This queue used to contain all the process in the ~~para~~ system.

- (2) Ready Queue:- This queue is used to contain process that are ready and waiting to be executed by the CPU.



Process Scheduling Diagram.

- A Scheduler is a system software component in the OS.
- It decides which process runs at what time.
- Why we need scheduler:-

- (1) Efficiently CPU usage.
- (2) Fast Response time
- (3) Balance load on the system.

Types of Schedulers:-

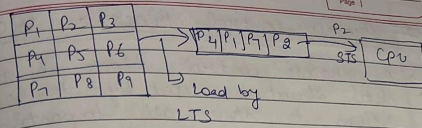
- (i) long term Schedulers
- (ii) Short term Schedulers
- (iii) medium term Schedulers

(1) long term Schedulers:- (LTS)

- The long-term scheduler is responsible for loading processes from disk into main memory so they can begin execution.
- Transfer processes from the job queue to ready queue.
- It is also called job scheduler.
- It is the slowest scheduler.

(2) Short term Schedulers (STS)

- It is responsible for selecting a process from the ready queue and assigning the CPU to it.
- This scheduler involved context switching.
- It is a CPU scheduler.
- Fast



(3) medium term Schedulers

- (i) It is a process-swapping scheduler.
- (ii) Its speed lies between long-term and short-term schedulers.
- (iii) It temporarily removes processes from main memory and stores them in secondary memory, also known as swapping.